

Anexo 11 — Runtime interactivo del seguimiento

Archivo: Scripts/run_interactive.py

Para que sirvio:

Version previa solo-cabeza: corre la policy de seguimiento con el objetivo controlado en tiempo real desde el navegador via WebSocket.

```
1 | """Modo interactivo: corre el robot con la policy v13 entrenada,
2 | target controlado en tiempo real desde un browser via WebSocket.
3 |
4 | Lanza dos cosas en paralelo:
5 | - Sim loop con MuJoCo viewer (thread principal).
6 | - Servidor FastAPI con WebSocket (thread daemon).
7 |
8 | Acceso local: http://localhost:8000
9 | Acceso LAN: http://<ip_de_la_pc>:8000 (mismo wifi, abrir puerto 8000 en firewall)
10 |
11 | Uso:
12 |     python Scripts/run_interactive.py
13 |     python Scripts/run_interactive.py --policy runs/ppo_dum_v13/final.zip --host 0.0.0.0 --port 8000
14 | """
15 |
16 | import argparse
17 | import io
18 | import queue
19 | import sys
20 | import threading
21 | import time
22 | from pathlib import Path
23 |
24 | sys.path.insert(0, str(Path(__file__).resolve().parent))
25 |
26 | import numpy as np
27 | import mujoco
28 | import mujoco.viewer
29 | from PIL import Image
30 | from stable_baselines3 import PPO
31 |
32 | from rl_env import DUMHeadTrackingEnv
33 | import web_remote.server as server
34 |
35 |
36 | class InteractiveEnv(DUMHeadTrackingEnv):
37 |     """Subclase del env de training que toma el target desde una cola
38 |     en vez de muestrearlo segun los modos pre-programados.
39 |     Nunca trunca: la sesion dura indefinidamente."""
40 |
41 |     def __init__(self, target_queue: queue.Queue, **kwargs):
42 |         # En modo interactivo el focus debe re-dispararse cada vez que el agente
43 |         # recupera el target (no solo una vez por sesion como en training).
44 |         kwargs.setdefault("one_shot_focus", False)
45 |         super().__init__(**kwargs)
46 |         self._target_queue = target_queue
47 |
48 |     def _update_target_position(self, sim_time):
49 |         """Override: target viene de la cola, no de las trayectorias predefinidas."""
50 |         try:
51 |             data = self._target_queue.get_nowait()
52 |             self._target_world = np.array(
53 |                 [float(data["x"]), float(data["y"]), float(data["z"])],
54 |                 dtype=np.float64,
55 |             )
56 |         except queue.Empty:
57 |             pass # mantener el target previo
58 |
59 |     def step(self, action):
60 |         obs, r, term, trunc, info = super().step(action)
61 |         return obs, r, term, False, info # nunca truncar en modo interactivo
62 |
63 |
64 | def run_uvicorn_in_thread(host: str, port: int):
65 |     """Lanza uvicorn en un thread daemon para no bloquear el sim loop."""
66 |     import uvicorn
67 |     config = uvicorn.Config(server.app, host=host, port=port, log_level="warning")
68 |     server_instance = uvicorn.Server(config)
69 |     th = threading.Thread(target=server_instance.run, daemon=True, name="uvicorn")
70 |     th.start()
71 |     return th
72 |
73 |
74 | def main():
```

```

75 | p = argparse.ArgumentParser()
76 | p.add_argument("--policy", type=str,
77 |               default=str(Path(__file__).resolve().parents[1] / "runs" / "ppo_dum_v13" / "final.zip"),
78 |               help="Ruta al .zip de la policy entrenada (default: v13).")
79 | p.add_argument("--host", type=str, default="0.0.0.0",
80 |               help="Bind host. 0.0.0.0 = accesible por LAN. 127.0.0.1 = solo local.")
81 | p.add_argument("--port", type=int, default=8000)
82 | p.add_argument("--target-smooth", type=float, default=0.3,
83 |               help="Coef LPF para suavizar target externo (0..1). 1=sin suavizado, 0.3=suave.")
84 | args = p.parse_args()
85 |
86 | # Colas de comunicacion sim <-> WS
87 | target_q = queue.Queue(maxsize=1)
88 | telemetry_q = queue.Queue(maxsize=1)
89 | frame_q = queue.Queue(maxsize=1) # JPEG bytes del render del viewer
90 | server.init_queues(target_q, telemetry_q, frame_q)
91 |
92 | # Lanzar servidor WS
93 | run_uvicorn_in_thread(args.host, args.port)
94 | print(f"[server] http://{args.host}:{args.port} (use http://127.0.0.1:{args.port} desde esta PC)")
95 | print(f"[server] desde celular en la misma red, usar la IP local del host")
96 |
97 | # Cargar env + policy
98 | env = InteractiveEnv(target_q)
99 | obs, _ = env.reset(seed=0)
100 | print(f"[sim] env OK, obs.shape = {obs.shape}")
101 | rl_model = PP0.load(args.policy)
102 | print(f"[sim] policy cargada: {args.policy}")
103 |
104 | # Renderer offscreen para el stream MJPEG a la web.
105 | # 480x480 / cada 2 steps de control = ~25 fps de video, suficiente.
106 | stream_renderer = mujoco.Renderer(env.model, height=480, width=480)
107 | frame_every_n_steps = 2
108 | frame_step_counter = 0
109 | print(f"[sim] renderer offscreen 480x480 listo (MJPEG stream en /stream)")
110 |
111 | # Lanzar viewer
112 | with mujoco.viewer.launch_passive(env.model, env.data) as viewer:
113 |     print(f"[sim] viewer abierto. Cerralo para terminar.")
114 |
115 |     target_smooth = float(np.clip(args.target_smooth, 0.0, 1.0))
116 |     sim_dt = float(env.model.opt.timestep) * env.frame_skip # tiempo entre steps de control (0.02 s)
117 |     last_fps_time = time.perf_counter()
118 |     fps_counter = 0
119 |     measured_fps = 50.0
120 |     telemetry_counter = 0
121 |
122 |     while viewer.is_running():
123 |         loop_start = time.perf_counter()
124 |
125 |         # Suavizado opcional del target (LPF de primer orden)
126 |         if target_smooth < 1.0 and not target_q.empty():
127 |             # Peek the queue, blend
128 |             try:
129 |                 new_data = target_q.queue[0] # peek
130 |                 new_target = np.array([new_data["x"], new_data["y"], new_data["z"]])
131 |                 env._target_world = (
132 |                     target_smooth * new_target
133 |                     + (1 - target_smooth) * env._target_world
134 |                 )
135 |                 # Replace in queue with smoothed value to avoid stale jumps
136 |                 try:
137 |                     target_q.get_nowait()
138 |                 except queue.Empty:
139 |                     pass
140 |                 target_q.put_nowait({
141 |                     "x": float(env._target_world[0]),
142 |                     "y": float(env._target_world[1]),
143 |                     "z": float(env._target_world[2]),
144 |                 })
145 |             except (IndexError, queue.Empty, queue.Full):
146 |                 pass
147 |
148 |         # Predict y step
149 |         obs = env._get_obs()
150 |         action, _ = rl_model.predict(obs, deterministic=True)
151 |         obs, _, _, _, info = env.step(action)
152 |
153 |         # Telemetria al cliente (10 Hz)
154 |         telemetry_counter += 1
155 |         if telemetry_counter >= 5:
156 |             telemetry_counter = 0
157 |             payload = {
158 |                 "theta_deg": float(info["theta_deg"]),
159 |                 "focus": bool(info["focus_triggered"]),

```

```

160 |         "fps": float(measured_fps),
161 |     }
162 |     try:
163 |         telemetry_q.get_nowait()
164 |     except queue.Empty:
165 |         pass
166 |     try:
167 |         telemetry_q.put_nowait(payload)
168 |     except queue.Full:
169 |         pass
170 |
171 |     # Render frame para el stream MJPEG (cada N steps para no matar el FPS)
172 |     frame_step_counter += 1
173 |     if frame_step_counter >= frame_every_n_steps:
174 |         frame_step_counter = 0
175 |         try:
176 |             stream_renderer.update_scene(env.data)
177 |             rgb = stream_renderer.render() # ndarray HxWx3 uint8
178 |             buf = io.BytesIO()
179 |             Image.fromarray(rgb).save(buf, format="JPEG", quality=75)
180 |             jpeg_bytes = buf.getvalue()
181 |             # Mantener solo el ultimo frame en la cola
182 |             try:
183 |                 frame_q.get_nowait()
184 |             except queue.Empty:
185 |                 pass
186 |             try:
187 |                 frame_q.put_nowait(jpeg_bytes)
188 |             except queue.Full:
189 |                 pass
190 |         except Exception as e:
191 |             # No romper el sim loop si falla el render por algun motivo.
192 |             print(f"[stream] render fallo: {e}")
193 |
194 |     # Sync viewer
195 |     viewer.sync()
196 |
197 |     # FPS measurement
198 |     fps_counter += 1
199 |     now = time.perf_counter()
200 |     if now - last_fps_time >= 1.0:
201 |         measured_fps = fps_counter / (now - last_fps_time)
202 |         fps_counter = 0
203 |         last_fps_time = now
204 |
205 |     # Mantener cadencia de 50 Hz (sim_dt = 20 ms)
206 |     elapsed = time.perf_counter() - loop_start
207 |     sleep_for = sim_dt - elapsed
208 |     if sleep_for > 0:
209 |         time.sleep(sleep_for)
210 |
211 |     print("[sim] viewer cerrado. Saliendo.")
212 |
213 |
214 | if __name__ == "__main__":
215 |     main()
216 |

```